Руководство по установке и развёртыванию

Документ описывает, как получить программу для ЭВМ «XML-библиотека „Берёста“ (beresta)», собрать её из исходного текста, установить готовые сборки, подключить к своему проекту и проверить работоспособность. Сведения относятся к версии, указанной в шапке документа.

Правообладатель: Ермаков Алексей Александрович. Сайт продукта: [адрес сайта продукта].

Содержание

1. Назначение и состав продукта
2. Где скачать
3. Системные требования
4. Сборка через Makefile
5. Сборка через CMake
6. Установка готовой сборки
7. Подключение к своему проекту
8. Проверка работоспособности
9. Обновление и удаление

Назначение и состав продукта

«Берёста» — библиотека для обработки XML на языке C++17 без сторонних зависимостей: разбор XML 1.0, DOM, SAX, пространства имён, восемь кодировок, DTD, XML Schema 1.0 (с утверждениями `xs:assert` и `xs:override` из XSD 1.1), XPath 1.0, XInclude, запись XML. Ядро собирается в динамическую библиотеку с плоским Си-интерфейсом, через который библиотекой пользуются программы на C, Java, C#, Python, Node.js и WebAssembly.

Результаты сборки:

Файл	Назначение
`beresta.dll` / `libberesta.so` / `libberesta.dylib`	динамическая библиотека с Си-интерфейсом (C, Java, C#, Python)
`libberesta.a` (Makefile) / `beresta_static` (CMake)	статическая библиотека для программ на C++ (полный C++ API)
`beresta_demo`, `beresta_capi_demo`	демонстрационные программы на C++ и на C
`beresta_tests`	модульные тесты
`beresta.node`	модуль Node.js (по флагу `NODE=1`)
`beresta_core.wasm` + `beresta.mjs`	сборка WebAssembly (`make wasm`, нужен Emscripten)

Из динамической библиотеки экспортируются только функции Си-интерфейса (71 функция). Программы на C++ подключают статическую библиотеку.

Где скачать

Что	Где
Исходный текст (репозиторий)	Gitverse: <https://gitverse.ru/ermaleyalekov/beresta>, зеркало GitHub: <https://github.com/ErmaleyAlekov/beresta>
Релизы: архив исходного текста и готовые сборки	Gitverse: <https://gitverse.ru/ermaleyalekov/beresta/releases>, GitHub: <https://github.com/ErmaleyAlekov/beresta/releases>
Пакеты для других языков	npm `beresta` и `beresta-wasm`, PyPI `beresta`, NuGet `Beresta`, Maven Central `io.github.ermaleyalekov:beresta`

В каждом релизе лежат архивы SDK для C и C++ `beresta-<версия>-<платформа>.zip` (каталог `include/`, динамическая и статическая библиотеки, `LICENSE`) и пакеты для других языков. Архив исходного текста релиза формирует сама площадка («Исходный код» на странице релиза).

Получить исходный текст через git:

```
git clone https://gitverse.ru/ermaleyalekov/beresta.git
```

Системные требования

Параметр	Требование
Процессор	x86_64 или ARM64 (aarch64); 32-битные платформы поддерживаются ядром, обёртки проверялись на 64-битных
ОС для сборки из исходного текста	Linux (в том числе Astra Linux, РЕД ОС, ALT), Windows 10 и новее, macOS 11 и новее. Проверочным стендом проверены Astra Linux SE 1.7 и 1.8, ALT p10 и p11, РЕД ОС 7.3 и 8
Готовые сборки Linux из релиза	glibc 2.35 и новее (собираются на Ubuntu 22.04). На системах со старой glibc (например, Astra Linux SE 1.7 — glibc 2.28) библиотеку нужно собрать из исходного текста
Компилятор	с поддержкой C++17: GCC 8 и новее, Clang, MinGW или MSVC. В CI проверяются GCC и Clang на Linux, MSVC на Windows, Apple Clang на macOS
Сборка	GNU Make (Makefile) или CMake
Сторонние библиотеки	не нужны: только стандартная библиотека C++17
Место на диске	около 100 МБ для исходного текста и результатов сборки (ориентировочно, без наборов тестов W3C)
Обёртки (по желанию)	Java 22+, .NET с поддержкой .NET Standard 2.1 (.NET Core 3.0 и новее), Python 3.8+, Node.js 18+ — только для соответствующих языков

Для сборки динамической библиотеки на Linux нужна статическая версия стандартной библиотеки C++ (`libstdc++.a`): библиотека встраивает её, чтобы на машине пользователя не требовалась определённая версия `libstdc++`.

Установка инструментов на российских ОС

Astra Linux, Debian, Ubuntu:

```
sudo apt-get install build-essential make
```

РЕД ОС (а также другие системы семейства RHEL):

```
sudo dnf install gcc-c++ make libstdc++-static
```

ALT Linux (пакет статической `libstdc++` называется по версии компилятора, например `libstdc++12-devel-static`):

```
apt-get install gcc-c++ make
```

CMake устанавливается пакетом `cmake` во всех трёх системах и нужен только для сборки через CMake.

> На GCC 8 (например, в Astra Linux SE 1.7) реализация `std::filesystem` находится в отдельной библиотеке `stdc++fs`: Makefile и CMake подключают её сами, если версия GCC ниже 9. Сборка и все модульные тесты проверены на Astra Linux SE 1.7 (GCC 8.3, glibc 2.28) через Makefile и через CMake.

Сборка через Makefile

Из корня исходного текста:

```
make
```

```
make test
```

Цель	Что делает
`make` / `make all`	`bin/libberesta.a`, динамическая библиотека, демо `bin/beresta_demo` (C++) и `bin/beresta_capi_demo` (C)
`make test`	собирает и запускает модульные тесты C++
`make ERRORS_LANG=en`	язык сообщений об ошибках по умолчанию (`ru`, если не задан)

Цель	Что делает
`make JAVA=1 CSHARP=1 PYTHON=1 NODE=1`	то же + бинарник в обёртки соответствующих языков
`make bindings-test`	самопроверка включённых обёрток
`make platform`	показать определённые ОС, архитектуру и пути копирования
`make clean` / `make fclean` / `make re`	удаление промежуточных файлов / всех результатов / полная пересборка

Поддерживается параллельная сборка (`make -j4`), компилятор можно сменить: `make CXX=clang++ CC=clang`. Объектные файлы и файлы зависимостей раскладываются по `obj/` и `dep/` зеркально `src/`; изменённый заголовок пересобирает только зависимые файлы.

Сборка через CMake

По умолчанию CMake собирает только динамическую библиотеку:

```
cmake -S . -B build
```

```
cmake --build build --config Release
```

Сборка с тестами и демонстрационными программами и запуск тестов:

```
cmake -S . -B build -DBERESTA_BUILD_TESTS=ON -DBERESTA_BUILD_EXAMPLES=ON
```

```
cmake --build build --config Release
```

```
ctest --test-dir build -C Release --output-on-failure
```

Опция	По умолчанию	Что делает
`BERESTA_BUILD_STATIC`	`OFF`	статическая библиотека `beresta_static` (цель `beresta::static`) для C++
`BERESTA_BUILD_EXAMPLES`	`OFF`	демонстрационные программы на C++ и C
`BERESTA_BUILD_TESTS`	`OFF`	модульные тесты и самопроверки включённых обёрток в `ctest`
`BERESTA_BUILD_JAVA`, `BERESTA_BUILD_CSHARP`, `BERESTA_BUILD_PYTHON`, `BERESTA_BUILD_NODE`	`OFF`	копирование бинарника в обёртки и цели упаковки

Опция	По умолчанию	Что делает
`BERESTA_ERRORS_LANG`	`ru`	язык сообщений об ошибках по умолчанию
`BERESTA_STATIC_RUNTIME`	`ON`	встроить C++ runtime в библиотеку

Для MSVC автоматически включаются ключи `/utf-8 /W4 /permissive-`.

Установка готовой сборки

1. Скачайте архив SDK для своей платформы со страницы релиза (Gitverse или GitHub).
2. Распакуйте его в каталог проекта, например `third_party/beresta`.
3. Заголовки лежат в `include/`, библиотеки — в каталоге архива для своей платформы. Динамическую библиотеку положите рядом с исполняемым файлом программы или в каталог, где её ищет загрузчик ОС (`PATH` в Windows, `LD_LIBRARY_PATH` или `rpath` в Linux).

Для других языков сборка не нужна — пакеты уже содержат бинарники всех платформ:

Язык	Установка
Node.js	`npm install beresta`
Браузер, Web Workers	`npm install beresta-wasm`
Python	`pip install beresta`
.NET	`dotnet add package Beresta`
Java	зависимость Maven `io.github.ermaleyalekov:beresta` (Gradle: `implementation("io.github.ermaleyalekov:beresta:2.1.3")`)

Подключение к своему проекту

Программа на C++ через CMake (исходный текст библиотеки — в подкаталоге проекта):

```
set(BERESTA_BUILD_STATIC ON CACHE BOOL "" FORCE)
add_subdirectory(third_party/beresta)
target_link_libraries(my_app PRIVATE beresta::static)
```

Программа на C++ вручную (в коде достаточно `#include "beresta.hpp"`):

```
g++ -std=c++17 -Ipath/to/beresta/include main.cpp path/to/beresta/bin/libberesta.a -o app
```

Программа на C через Си-интерфейс (`#include "capi/beresta_c.h"`):

```
gcc -std=c99 -Ipath/to/beresta/include app.c -Lpath/to/beresta/bin -lberesta -o app
```

Цель CMake `beresta::beresta` — динамическая библиотека для программ на C.

Проверка работоспособности

После сборки через Makefile:

```
make test
```

Ожидаемый итог — строка вида `Итого: N тестов, пройдено N, упало 0`. Демонстрационные программы `bin/beresta_demo` и `bin/beresta_capi_demo` показывают разбор, XPath, построение документа и проверку по схеме.

Проверочный стенд одной командой собирает библиотеку из исходного текста, проверяет поддержку C++17 и наличие статической `libstdc++`, запускает модульные тесты и демонстрационные программы и записывает отчёт `stand_report.txt` (журналы шагов — в `stand_logs/`):

```
sh tools/stand/run_stand.sh
```

Подробнее о стенде — в файле `tools/stand/README.md`.

Обновление и удаление

- Обновление: скачайте новую версию (или выполните `git pull`) и пересоберите (`make re` или новая сборка CMake). Изменения версий — в файле `CHANGELOG.md`.
- Удаление результатов сборки: `make fclean` (или удалите каталог сборки CMake). Библиотека не устанавливает файлов в системные каталоги и не меняет настроек ОС: достаточно удалить её каталог.